

operating system is compatible with both the display shapes and thus the platform gives you the option to choose the shape and the way you would want to choose the code to load up differently depending on the shape or just a special layout compatible with both.

In android studio, The **WatchViewStub** class included in the Wearable UI Library lets you specify different layout definitions for square and round screens. This class detects the screen shape at runtime and gets you the corresponding layout.

A **WatchViewStub** class depends on the reports/feedback from the on **ApplyWindowsInsets** class. If the call is not delivered by initial measurements, the container will be assumed rectangle by default. To use this class for handling different screen shapes in your app, you would need to Add **WatchViewStub** as the main element of your activity's layout and then specify a layout definition file for square screens with the **rectLayout** attribute or specify a layout definition file for round screens with the **roundLayout** attribute.

After defining these class and attributes create different layout definition files for square and round screens. You define these layouts in the same way that you create layouts for handheld apps, but taking into account the constraints of wearable devices. The system inflates the correct layout at runtime depending on the screen shape.

Another aspect that needs to be considered and well decided upon needs to be navigation between screens of the app. If the app has more than one screen, then would it be having buttons to navigate back and forth or would it be a slide. This would greatly depend on the App structure. Read about the Android wear app structures here (<https://developer.android.com/design/wear/structure.html#2DPicker>) and for Apple WatchKit instructions refer: <http://code.tutsplus.com/tutorials/watchkit-navigation-transitions-and-contexts--cms-23938>

These smartwatches light up only when they are raised to eye level which is the interactive mode and the rest of the time hibernate in the ambient mode. It is important to note that you need to define the app status, design and structure in both the modes.

There are some other additional instructions mentioned by the Android wear developers stated here: <https://developer.android.com/design/wear/watch-faces.html>

Developing the app

In this section we will have a look at the actual development of your app.